LLM-Text Watermarking based on Lagrange Interpolation



Manoj Kumar V K

Under the Guidance of

Dr. Shibam Ghosh

Department of Computer Science
Indian Insitute of Technogology Hyderabad

1. Introduction
2. Methodology
3. Extraction Variants
4. Security Analysis
5. Evaluation
6. Extensions
7. Conclusion
8. References

Introduction

- LLM-generated text can be modified, paraphrased, truncated, or partially deleted.
- Such modifications can disrupt watermark detection and authorship attribution.
- Watermarking embeds identifiable information into generated text without requiring a visible signature.
- **Problem:** Recover the embedded author identity even when only part of the original text survives.

Research Gaps

- Recovering a specific author's identity is more demanding than simply detecting a watermark.
- Text modifications can corrupt or remove embedded watermark information.
- The verifier must distinguish genuine watermark points from corrupted or spurious points.

Proposed Watermarking Scheme

1. Encode the secret identity as coefficients of a polynomial over $GF(2^n)$.
2. Generate pseudorandom coordinates x_i using a secret-seeded generator.
3. Compute watermark points $(x_i, f(x_i))$.
4. Embed the bits of these points into the generated text by biasing token-selection logits.
5. Recover candidate points from the text and use Maximum Collinear Points (MCP) to identify the dominant line.
6. Reconstruct the secret identity from the line's coefficients.

Algorithm 1: Watermark Embedding Secret Identity K

Input: Autoregressive language model **LLM** used to generate a text of length $T = 2nN$, prompt sequence PR, token set $\mathcal{V}$, n-bit secret key s_k, LFSR with n-bit state and straight line $f(x) = a_0 + a_1 x$ with the secret identity $K = (a_0 \| a_1)$, where computations are done in $GF(2^n)$.

Output: Public LLM text $S_{1,1}, \ldots, S_{N,2n}$.

Initialize $LFSR(t_0, s_k)$
Using secret key s_k, create two lists $\mathcal{V}_0$, $\mathcal{V}_1$, where $\mathcal{V}_i$ are tokens carrying watermark $i \in \{0, 1\}$ ($\mathcal{V}_0 \cup \mathcal{V}_1 = \mathcal{V}$)
for $i = 1$ to N **do**
    $x_i = (x_{i,1}, x_{i,2}, \ldots, x_{i,n}) \leftarrow LFSR(t_i, s_k)$
    $y_i = f(x_i) = (y_{i,1}, y_{i,2}, \ldots, y_{i,n})$
    for $j = 1$ to n **do**
        Compute logits: $v_1 = \text{LLM}(PR, S_{1,1}, \ldots, S_{i,2(j-1)})$
        Bias the logits:
        if $x_{i,j} = 0$ **then** add bias to $\hat{v}_1[w] = v_1 + \delta$ for all $w \in \mathcal{V}_0$
        else add bias to $\hat{v}_1[w] = v_1 + \delta$ for all $w \in \mathcal{V}_1$
        end if
        Append $S_{i,2j-1}$ to S by sampling from biased logits $\hat{v}_1$
        Compute logits: $v_2 = \text{LLM}(PR, S_{1,1}, \ldots, S_{i,2j-1})$
        Bias the logits:
        if $y_{i,j} = 0$ **then** add bias to $\hat{v}_2[w] = v_2 + \delta$ for all $w \in \mathcal{V}_0$
        else add bias to $\hat{v}_2[w] = v_2 + \delta$ for all $w \in \mathcal{V}_1$
        end if
        Append $S_{i,2j}$ to S by sampling from biased logits $\hat{v}_2$
    end for
end for
return S

Algorithm 2: Watermark Extraction

1: **Input:** LLM text $\widetilde{\mathcal{T}} = (\tilde{S}_{1,1}, \ldots, \tilde{S}_{N,2n})$, token set $\mathcal{V}$, n-bit secret key s_k, n-bit state LFSR, which generates x-coordinates of unknown $f(x)$, where points $(x_i, f(x_i))$ are embedded into the text according to Algorithm 1; $i = 1, \ldots, N$, computations are done in $GF(2^n)$.

2: **Output:** The secret user identity $K = (a_0 \| a_1)$

3: Create an empty list of points $\mathbb{P} = \varnothing$ of $(x, f(x))$

4: Initialize $LFSR(t_0, s_k)$

5: Create the sequence $\mathbb{L} = (x_1, \ldots, x_N)$, where $x_i = LFSR(t_i, s_k)$

6: Using secret key s_k, create two lists $\mathcal{V}_0$, $\mathcal{V}_1$, where $\mathcal{V}_i$ are tokens carrying watermark $i \in \{0, 1\}$ $(\mathcal{V}_0 \cup \mathcal{V}_1 = \mathcal{V})$

7: Extract watermark $\mathbb{W} = (w_1, \ldots, w_{2nN})$ using $\mathcal{V}_0$ and $\mathcal{V}_1$

8: **for** $i = 1$ to N **do**

9: take x_i

10: **while** $x_i \neq (w_j, \ldots, w_{j+n})$ **do**

11: move x_i along $\mathbb{W}$ bit by bit for $j = 1, \ldots, n(2N-1)$

12: **end while**

13: **if** $x_i = (w_j, \ldots, w_{j+n})$ **then**

14: $\mathbb{P} \leftarrow \mathbb{P} \cup (x_i, (w_{j+n+1}, \ldots, w_{j+2n}))$

15: **end if**

16: **end for**

17: **return** $\tilde{K} = (a_0 \| a_1)$ if the set $\mathbb{P}$ contains at least $\varepsilon\ell$ points on the straight line $f(x) = a_0 + a_1 x$

Maximum Collinear Points (MCP)

- Text modification may produce both genuine and corrupted watermark points.
- Genuine points lie on the original line:

$$f(x) = a_0 + a_1 x$$

- The MCP problem finds the largest subset of points lying on a common straight line.
- The line with the largest number of recovered points is considered the watermark candidate.

Simplified Watermark Extraction

- Slide a $2n$-bit window across the extracted watermark bit sequence.
- Interpret each window as a candidate point.
- Apply MCP directly to all candidate points.
- This avoids filtering candidates using valid LFSR-generated coordinates but introduces more spurious points.

Double-Resolution Watermarking

- Consecutive points can be constructed using repeated applications of the linear function:

$$f(x) = a_0 + a_1 x$$

$$(x, f(x)),\ (f(x), f^2(x)),\ \ldots,\ (f^{N-1}(x), f^N(x))$$

- Consecutive points share coordinates, reducing the number of candidate points produced by sliding windows.
- Candidate count decreases from $2n(N-1)+1$ to $n(N-1)+1$.
- Short cycles may require a new initial value; the security implications must also be considered.

Security Analysis: Watermark Recovery

- Let $N = F + R$, where:
  - F: genuine points lying on the secret line.
  - R: random or corrupted points.
- The analysis models corrupted points as randomly distributed over $GF(2^n) \times GF(2^n)$.
- Recovery is reliable when the genuine line contains more points than any competing line formed by corrupted points.
- Larger finite fields reduce the probability of accidental collinearities under this model.

$$f(x) = a_0 + a_1 x, \qquad K = (a_0 \parallel a_1)$$

Algebraic Attack and Countermeasures

- **Vulnerability:** An LFSR and fixed token partitions can expose linear relationships in the watermark construction.
- **Potential attack:** An adversary may construct linear equations from observed token occurrences to infer the LFSR seed and watermark bits.
- **Countermeasures**
  - Replace the LFSR with a nonlinear feedback shift register (NFSR).
  - Use dynamic token partitions rather than fixed V_0 and V_1.
  - Derive partitions using cryptographic hashes and the secret key.
- Stronger security may introduce additional computational overhead.

Security Analysis: Probability Bound

Theorem: *Given a plane $GF(2^n) \times GF(2^n)$, and R random points, the probability that there is a straight line that contains k points is approximated by*

$$\binom{R}{k} \left(\frac{1}{2^n}\right)^{k-2} \left(1 - \frac{1}{2^n}\right)^{R-k}.$$

Experimental Setup

- **Objective:** Evaluate the maximum number of accidental collinear points in random point sets.
- **Method:** Generate random points over finite fields of different sizes.
- **Fields:**

$$GF(2^6),\ GF(2^8),\ GF(2^{12}),\ GF(2^{16}),\ GF(2^{32})$$

- **Evaluation:** Measure the maximum number of collinear points for varying numbers of random points.

Experimental Results

Random Points	$GF(2^6)$	$GF(2^8)$	$GF(2^{12})$	$GF(2^{16})$	$GF(2^{32})$
8	3	2	2	2	2
16	3	3	2	2	2
32	5	3	2	2	2
64	6	4	3	2	2
128	6	5	3	2	2
256	10	5	3	2	2
512	14	8	3	2	2
1024	20	11	4	2	2

- Small fields exhibit more accidental collinearities.
- For $GF(2^{16})$ and $GF(2^{32})$, the observed maximum remains two in these experiments.
- Larger fields make accidental watermark-like structures less likely in the tested random sets.

Multiple Secrets and Higher-Degree Polynomials

- **Multiple lines as secrets**
  - Embed multiple lines, each encoding a distinct secret.
  - Recover the top t lines using MCP.
  - Establish a deterministic ordering before concatenating the secrets.
- **Higher-degree polynomials**

$$f(x) = a_0 + a_1 x + \cdots + a_d x^d$$

  - Encode the secret in the polynomial coefficients.
  - Reconstruct the polynomial using Lagrange interpolation.
  - Identify genuine points using Maximum Co-Polynomial Points (MCPP).

Conclusion and Future Work

- **Conclusion:**
  - Proposes Lagrange interpolation-based watermarking to encode author identity in LLM-generated text.
  - Uses MCP to recover the watermark from potentially modified text.
  - Improves security using cryptographically stronger pseudorandom generation and dynamic token partitions.
  - Supports extensions to multiple secrets and higher-degree polynomials.
- **Future work:**
  - Improve resilience against adversarial text modifications.
  - Optimize recovery of multiple secrets.
  - Explore hybrid schemes combining lines and higher-degree polynomials.

References


Jarosław Janas, Paweł Morawiecki, and Josef Pieprzyk.
Llm-text watermarking based on lagrange interpolation, 2026.

Thank You